

Team Security Sharing

OWASP 安全编码规范 与 AI 时代的安全落地

从传统 Web 安全领域，走到 Top 10:2025、Agent / MCP / Skill 风险，再走到团队里真正可试点的 AI 安全能力。

受众

前端 / 后端 / 运维平台

技术栈

Java + Spring Boot + MySQL

主线

规范 -> 风险 -> AI -> 落地

安全分享最怕的，不是大家没听过名词

而是大家知道风险，却没有把它变成默认动作

今天这 1 小时，我们想解决什么

希望带走的东西

不是更多术语，而是更稳定的默认动作

- 写接口时想到输入校验、权限校验、异常处理、日志审计
- 写页面时想到输出编码、富文本风险、文件上传风险
- 做部署时想到默认配置、暴露面、凭据、补丁
- 用 AI 时想到 prompt、tool、token、skill、供应链

分享主线

- 1 从 `OWASP_zh.pdf` 看安全编码底线
- 2 用 `OWASP Top 10:2025` 对齐现代高频风险
- 3 看看 AI 时代，OWASP 已经变成了什么样
- 4 讨论怎样把 OWASP 嵌入团队日常流程和 AI 助手里

第一部分

OWASP_zh.pdf 依然值得讲，因为它提供的是底线

为什么这份中文 PDF 今天仍然有价值

它不是某个框架教程

它更像一份跨技术栈的底线规则集：输入、输出、认证、会话、授权、日志、配置、数据库、文件上传.....

不管技术怎么变，Web 开发总有一些必须反复做到的安全常识。

输入

所有输入都不可信

输出

所有输出都要按上下文编码

权限

认证、会话、授权必须在服务端收口

交付

日志、配置、数据库、文件都属于安全边界

四个总原则

- **不要把客户端限制当成安全控制** 前端校验、隐藏字段、按钮灰掉只能改善体验
- **安全控制要集中化、服务端化、可复用** 不要把安全逻辑散在每个 Controller 里
- **默认拒绝，而不是默认放行** 特别是权限、配置、异常处理和文件访问
- **安全问题通常是“链路没收口”** 页面、服务端、数据库、日志、部署必须一起看

今天第一部分，我只建议团队牢牢记住六个主题

输入验证 / 输出编码 / 认证与会话 / 访问控制 / 错误与日志 / 配置数据库文件

输入验证：所有输入都不可信

“在可信系统（比如：服务器）上执行所有的数据验证。”

“尽可能采用‘白名单’形式，验证所有的输入。”

“在处理以前，验证所有来自客户端的数据……”

对团队最重要的提醒

- 输入不只是表单字段，还包括 URL、JSON、Header、Cookie、MQ、回调、上传文件名
- 校验不只是“非空”，还要覆盖类型、长度、范围、枚举值、业务合法性
- 数据库和缓存里再次读出来的数据，也可能是不可信的

输入验证：最常见的三类事故

真实高频场景

- 接口只校验非空，不校验状态流转是否合法
- 排序字段、表名动态传入，最后拼进 SQL
- 文件下载接口接受路径参数，出现路径穿越

落地动作

- DTO 层做注解校验，业务层做规则校验
- 排序字段、状态流转、模板名、对象 key 全部白名单化
- 统一拦截器 / 网关 / 公共组件承接可复用规则

输出编码：不是“过滤脏字符”，而是“按上下文编码”

“通过语义输出编码方式，对所有返回到客户端的来自于应用程序信任边界之外的数据进行编码。”

关键认知

同一份用户输入，落在 HTML 文本、HTML 属性、JS 片段、URL、响应头里，需要的处理方式都不一样。

常见误区

- 以为统一替换 `<` 和 `>` 就够了
- 以为 React / Vue 默认转义就能覆盖所有 XSS 场景
- 忘记 `v-html`、富文本、Markdown、下载文件名回显

落地动作

- 前端默认使用安全渲染能力，少做原始 HTML 注入
- 富文本统一做白名单清洗
- 后端输出到不同上下文前，选择对应编码策略

认证与会话：别把“登录成功”当成安全终点

“对所有的网页和资源要求身份验证。”

“身份验证的失败提示信息应当避免过于明确。”

“注销功能应当完全终止相关的会话或连接。”

要按整条链路设计

- 密码怎么存，是否抗弱密码和撞库
- 是否有频控、锁定、MFA、风险登录控制
- Token 生命周期多长，能不能撤销
- 登出后是否真的失效，切换账号后旧会话是否还有效

访问控制：最容易真的出事故的一章

“只使用可信系统对象.....以做出访问授权的决定。”

“在每个请求中加强授权控制.....”

“限制只有授权的用户才能访问直接对象引用。”

常见错误

- 只做登录校验，没有做授权校验
- 只做页面权限，没有做接口权限
- 只做功能权限，没有做数据权限
- 改一下 `orderId` / `userId` 就能看别人的数据

记忆句

权限控制至少要回答三个问题：你是谁、你能做什么、这份数据是不是你的。

错误处理与日志：对外要收敛，对内要完整

“不要在错误响应中泄露敏感信息.....”

“日志记录控制应当支持记录特定安全事件的成功或者失败操作。”

常见问题

- 堆栈信息直接返回前端
- 日志打出 token、手机号、身份证、请求体
- 登录失败和权限失败没有审计

平衡点

- 对外：错误信息尽量抽象
- 对内：关键安全事件必须可追踪、可告警、可复盘
- 日志里不能再制造新的泄露

配置、数据库与文件上传：很多事故不在业务代码里

“确保服务器、框架和系统部件安装了当前使用版本的所有补丁。”

“使用强类型的参数化查询方法。”

“只验证文件类型扩展是不够的。”

团队高频落点

- Actuator、Swagger、测试端点在生产暴露
- 数据库账号权限过大，MyBatis `${}` 动态拼接
- 上传目录可执行，对象存储桶默认公开
- 只看扩展名，不看文件头

第一部分只收束成一句

安全不是上线前扫一次，而是需求到部署全过程都要守住默认底线

第二部分

OWASP Top 10:2025 告诉我们，现代风险已经不只在代码里

Top 10:2025 总览

官方排名

1. A01 Broken Access Control
2. A02 Security Misconfiguration
3. A03 Software Supply Chain Failures
4. A04 Cryptographic Failures
5. A05 Injection
6. A06 Insecure Design
7. A07 Authentication Failures
8. A08 Software or Data Integrity Failures
9. A09 Security Logging & Alerting Failures
10. A10 Mishandling of Exceptional Conditions

2025 版最值得注意的三点

- **配置错误** 升到第 2 位：暴露面和默认配置问题非常现实
- **软件供应链** 升到第 3 位：安全边界早就不只在业务代码里
- **异常条件处理不当** 进入 Top 10：失败路径正式成为攻击面

我建议重点讲的六项

A01

Broken Access Control

A03

Software Supply Chain
Failures

A06

Insecure Design

A02

Security Misconfiguration

A05

Injection

A10

Mishandling of
Exceptional Conditions

为什么这样取舍

- A01：我们最容易真的出事故
- A02 / A03：现代工程现实最强
- A05：最经典，也最容易被误判“已经解决”
- A06：把安全前移到设计阶段
- A10：2025 新信号，失败路径也是攻击面

A01 Broken Access Control

核心问题

用户能不能访问不属于自己的资源， 或执行本不该执行的操作。

典型事故

改 ID 看别人数据、接口漏权限、导出越权、多租户隔离只做在页面上。

治理抓手

- 统一身份、统一授权模型
- 关键对象的 ownership 校验
- 接口层和数据层都要控权
- 增加越权专项测试

越权问题本质上是：业务边界没有被系统真正执行。

A02 Security Misconfiguration

今天为什么这么靠前

- 默认配置没关
- 端点、页面、服务暴露过多
- 错误页暴露细节
- 安全头缺失
- 对象存储、容器、网关没有做收口

对团队的现实要求

- 基线模板化
- 暴露面盘点
- 自动检查而不是人工记忆
- 上线前配置核查

这类问题通常不是“谁写错了一行代码”，而是整个交付链路缺少统一安全基线。

A03 Software Supply Chain Failures

今天的供应链不只等于依赖版本

- Maven 依赖和传递依赖
- 镜像来源与制品仓库
- CI/CD 权限与发布流程
- 插件、脚本、自动化工具

团队现实动作

- 做依赖和镜像清单
- 关注高危漏洞公告
- 构建产物可追溯
- 关键仓库和流水线做最小权限与 MFA
- 逐步引入 SBOM、签名和来源证明

哪怕业务代码写得很好，交付链路被污染，结果仍然可能是不安全的。

A05 Injection

不只是 SQL 注入

- SQL / HQL / JPQL 注入
- 命令注入
- 表达式注入
- 模板注入
- 未来 LLM 场景下的 Prompt Injection

记忆方式

注入的本质，是把 **不可信输入** 当成了 **程序指令的一部分**。

- 能结构化传参就不要字符串拼接
- 能不用解释器式接口就不用
- 必要时对白名单控制可变部分
- 审计 MyBatis `${}`、shell、模板、表达式引擎

A06 Insecure Design + A10 Exceptional Conditions

A06 不安全设计

很多严重问题不是因为代码写错，而是因为设计阶段根本没把攻击者放进场景里。

例子

弱密码找回、审批绕过、多租户隔离先天薄弱、抢购和领券没有防滥用设计。

A10 异常条件处理不当

失败路径本身就是攻击面。参数缺失、超时、回退、空值、降级、并发竞争都可能绕开安全控制。

记忆句

关键安全能力在异常情况下必须 fail-secure，而不是 fail-open。

其余四项，用短讲方式带过

A04 算法、密钥、传输链路和用错场景

A07 弱密码、撞库、找回密码、MFA、token 生命周期

A08 来源和完整性验证不足，信任了不该信任的制品或数据

A09 安全事件没有被记录、发现和告警

第二部分真正想传达的三层变化

- 经典风险仍在，尤其是越权、注入、认证
- 工程化风险越来越重要，尤其是配置、供应链、完整性、日志
- 失败路径已经正式成为攻击面，异常处理不能再只当稳定性问题看

第三部分

AI 时代，OWASP 已经不只是“经典 Web Top 10”

An error occurred on this slide. Check the terminal for more information.

变化一：LLM Top 10 升级为 GenAI Security Project

这意味着什么

OWASP 不再把 AI 安全理解成“给 LLM 列 10 条风险”。它已经开始建设一整套持续治理体系。

对团队的启发

- 只要项目接入 LLM，就已经进入新的安全面
- 问答、RAG、代码助手、文档处理、Agent 都算
- 以后关注的不只是“幻觉”，还包括输入面、输出面、权限面和成本面

变化二：MCP、Skill、NHI 是新一代高相关名词

MCP Top 10

看 AI 系统怎样连接工具和外部系统

Agentic Skills Top 10

看 AI 被“教会怎么做事”的技能模板本身

Non-Human Identities Top 10

看不是人、但权限很大的 token / service account / robot

为什么这和我们团队强相关

- AI 助手会持有 token、文件权限、工具调用能力
- skill 像“可执行流程模板”，会变成新的供应链和执行面
- 非人身份会越来越多，且权限常常比普通用户更大

AI 时代的 OWASP，正在同时覆盖

开发 / 平台 / 安全治理 / 标准对齐

第四部分

让 AI 增强 OWASP 在项目中的参与度和使用便捷性

最值得做的，不是“让 AI 负责安全”

而是让 AI 做两件事

- 把安全能力前移
- 把安全规范嵌入日常研发动作

需求评审、方案评审、代码评审、测试点设计、上线检查，才是最适合让 AI 帮忙的地方。

核心判断

最值得做的不是再写一份规范，而是让 OWASP 通过 AI 和工程化能力，进入团队每天都在重复的动作。

OWASP agent skill 可以长什么样

四种工作模式

- 需求评审模式：看越权、认证、数据保护、审计缺口
- 方案评审模式：看不安全设计、信任边界、权限模型
- 代码评审模式：看注入、日志泄露、文件上传、配置错误
- 上线检查模式：看默认配置、暴露端点、弱凭据、缺日志

建议的统一输出

- 命中的 OWASP 类别
- 风险原因与可能攻击方式
- 修复建议
- 推荐测试点
- 严重级别
- 是否需要人工复核

An error occurred on this slide. Check the terminal for more information.

最小可行试点建议

第一版先覆盖 8 到 12 条高频规则

- 越权
- SQL 注入
- XSS / 输出编码
- 文件上传
- JWT 生命周期
- 敏感信息日志
- 密钥硬编码
- 默认配置暴露
- 异常 fail-open

试点方式

- 选一个典型业务模块
- 把 skill 输出和现有 code review 结果做对比
- 把效果好的规则沉淀成团队自己的规范和模板

AI 输出最终要反哺制度资产，而不是只停留在一次对话里。

今天最值得带走的四句话

1. OWASP_zh.pdf 提供的是今天仍然有效的安全底线。
2. Top 10:2025 告诉我们，现代风险已经扩展到配置、供应链、异常处理和完整性。
3. AI 时代的 OWASP 已覆盖 LLM、Agent、MCP、Skill、非人身份和治理体系。
4. 最值得做的是把 OWASP 通过 AI 和工程化能力嵌入日常研发流程。

结束句

如果规范永远躺在文档里，它只能靠记忆执行；

如果规范进入评审、进入代码、进入流水线、进入 agent，它才开始真正改变项目。

Thanks

Q & A

可以从三个方向继续聊：

团队里最容易出现的越权和配置问题

如果要试点一个 OWASP agent skill，第一版怎么做

哪些 AI 安全风险最值得提前纳入项目规范